

QRCS Corporation

Quantum Message Authentication Code Analysis

Title: Implementation Analysis of the Quantum Message Authentication Code (QMAC)

Author: John G. Underhill

Institution: Quantum Resistant Cryptographic Solutions Corporation (QRCS)

Date: November 2025

Document Type: Technical Cryptanalysis Report

Revision: 1.0

Chapter 1: Introduction

The Quantum Message Authentication Code (QMAC) is a deterministic, polynomial based message authentication construction built on cSHAKE derived subkeys and multiplication in a binary field of size 2^{256} . Although its design is conceptually straightforward, its correct operation depends on the precise interplay between its initialization procedure, the field arithmetic used in its universal hashing layer, and the final masking step that protects internal accumulator state. This document provides an implementation focused examination of QMAC as realized in the QSC library, with the goal of ensuring that the code accurately reflects the intended design, maintains mathematical correctness, and meets modern expectations for cryptographic engineering quality.

The purpose of this document is threefold. First, it establishes a clear and accurate description of what QMAC is intended to do, how it derives its subkeys, how it processes messages, and how it produces authentication tags. Second, it examines the implementation line by line to verify that it faithfully instantiates these behaviors without introducing deviations, unnecessary complexity, or hidden assumptions. Third, it evaluates the implementation through the lens of security relevant engineering concerns, including constant time properties, memory handling, determinism, and lifecycle management.

This chapter provides the conceptual grounding for the document. QMAC is not a general-purpose cryptographic framework but a narrowly scoped primitive that performs one task: producing a deterministic authentication tag from a secret key,

optional domain separation parameters, and a message. The construction derives two internal subkeys through a single cSHAKE operation. The first subkey drives a polynomial hash over 256-bit message blocks, and the second masks the final hash result to produce the output tag. Unlike keyed hash constructions that rely on repeated invocations of a hash function, QMAC operates with a tight and minimal structure, which makes correctness of each internal step essential.

The remainder of this document is organized to reflect the operational sequence of QMAC. After presenting a concise summary of the construction, it examines the specification intent, the code architecture, and each of the core operational phases: subkey derivation, message processing, field arithmetic, internal function behavior, alignment with the mathematical model, and security relevant implementation properties. These chapters build toward a complete and coherent view of how QMAC behaves in practice and whether it satisfies its design objectives. The final chapters address verification, reproducibility, limitations, and opportunities for future engineering refinement.

Chapter 2: Construction Summary

2.1 Overview

QMAC is a deterministic message authentication code that combines cSHAKE-based subkey derivation with polynomial evaluation in a large binary field. The construction is compact, consisting of an initialization phase that derives two internal subkeys, a message processing phase that updates a field-based accumulator, and a finalization phase that masks the accumulator before producing the authentication tag. This chapter provides a high-level description of these components as they are intended to operate.

The emphasis here is conceptual clarity: the goal is to explain the construction's purpose and structure without reference to implementation details, which are examined later in the document.

2.2 Subkey Derivation Model

QMAC begins with a single cSHAKE invocation that absorbs a secret key and optional domain-separation parameters. The function cSHAKE is well suited for this task because

it provides customizable hashing behavior without requiring repeated keyed or unkeyed permutations.

Two internal 256-bit subkeys are extracted from the first block of cSHAKE output:

- H , the hashing key used for polynomial accumulation
- F , the finalization key used to mask the accumulator

These subkeys define the behavior of QMAC for the lifetime of a single MAC computation. Their independence is a direct consequence of the pseudo-randomness provided by cSHAKE and the fact that extraction uses disjoint, contiguous regions of its output.

2.3 Polynomial Hashing Process

The message is processed in fixed-length blocks of 32-bytes, each interpreted as an element of the field $GF(2^{256})$. The accumulator Y , which begins at zero, evolves according to a linear recurrence that combines XOR and field multiplication.

2.3.1 Block Encoding

Each 32-byte message segment is converted into a field element directly through byte-to-bit interpretation. The final block is padded with zeros if it contains fewer than 32 bytes. This simple encoding avoids ambiguity and ensures that equal messages always produce equal encodings.

2.3.2 Accumulator Recurrence

For each block X , the accumulator is updated using

$$Y \leftarrow H \cdot (Y \oplus X).$$

This recurrence forms the core of the polynomial MAC construction. It ensures that each block influences the final value in a structured, algebraically meaningful way. Because the field is defined by an irreducible pentanomial modulus, multiplication and reduction behave exactly as required for a secure universal-hash family.

2.4 Finalization and Tag Formation

Once all message blocks have been processed, the accumulator contains a field element representing the polynomial evaluation of the encoded message. However, the accumulator itself is not returned to the caller. Instead, QMAC applies a final mask:

$$T = Y \oplus F.$$

This masking step ensures that internal accumulator values cannot be recovered by examining tags. It also prevents certain forms of algebraic manipulation that might arise if the raw accumulator were exposed across multiple messages or protocol contexts.

The final tag T is a deterministic 32-byte value that depends on the key, the absorbed customization parameters, and the message.

2.5 Summary

The construction of QMAC can be understood as a three-phase process:

1. **Subkey derivation via cSHAKE**, producing two 256-bit keys.
2. **Polynomial accumulation**, in which each block contributes to a growing field element under multiplication by a secret, fixed value.
3. **Final masking**, which protects internal state and yields the output tag.

This design is compact, algebraically coherent, and amenable to efficient verification. The next chapter examines how the specification's intended behavior aligns with the code, setting the stage for a detailed implementation analysis.

Chapter 3: Specification Alignment and Design Intent

3.1 Overview

This chapter examines the intended design of QMAC as defined by the algorithm's conceptual specification and compares it to the structural requirements that guide the implementation. The goal is to clarify what QMAC is supposed to accomplish, how its components are expected to behave under the model assumed by the designer, and what invariants must hold for the construction to remain secure and correct.

Unlike the implementation chapters that follow, this section focuses on the *idealized* view of QMAC, independent of code structure or engineering decisions. Establishing this foundation is essential for later verification of correctness and alignment.

3.2 Intended Algorithmic Behavior

The design intent of QMAC centers on three linked behavioral properties: determinism, structural simplicity, and algebraic correctness.

The construction is deliberately minimal. It uses a single call to cSHAKE to derive all necessary subkeys, a single accumulator that evolves linearly under field operations, and a final masking step to protect internal state. These components work together to produce a deterministic authentication tag whose security derives from both the unpredictability of the subkeys and the algebraic difficulty of forging a valid tag under an unknown hashing key.

Determinism is an explicit design feature. QMAC does not incorporate randomness into tag generation. For a given key and domain-separation context, equal messages always result in equal tags, and unequal messages never share encodings. Protocols that require nondeterministic behavior must wrap QMAC in a larger construction.

3.3 Domain Separation and Customization Intent

The specification assumes that cSHAKE will be used for more than simple key stretching. Its customizable nature enables callers to establish independent cryptographic domains. The presence of the nonce and info fields allows protocol designers to bind QMAC to specific message types, application layers, transport channels, or protocol states.

3.3.1 Intended Use of Nonce

The nonce is not intended to randomize the MAC output. Instead, it is designed to act as a structural input to distinguish one context from another. Two protocol states that must not share a QMAC instance can absorb different nonces, even with the same secret key.

3.3.2 Intended Use of Info

The info field supports additional binding of metadata to the MAC state. The designer's intention is that protocol-level semantics; such as message type, version, or channel, can be encoded into the sponge during initialization.

These customization fields form part of the design intent and are central to ensuring that QMAC instances do not collide across different usage contexts.

3.4 Field Selection and Algebraic Requirements

A key aspect of the intended model is the behavior of the polynomial hashing component. QMAC requires a field of size 2^{256} to ensure strong collision bounds and a security margin consistent with modern cryptographic expectations. For the polynomial evaluation to function properly as a universal-hash family, the modulus defining the field must be irreducible.

The irreducible pentanomial

$$m(x) = x^{256} + x^{10} + x^5 + x^2 + 1,$$

satisfies this requirement. It ensures the existence of multiplicative inverses for all nonzero elements and prevents structural weaknesses such as those introduced by reducible moduli.

This algebraic requirement is fundamental to the design intent of QMAC's hashing layer.

3.5 Implementation-Level Invariants

The specification expects several invariants to hold, regardless of platform or architectural details:

1. **Accumulator Initialization**

The accumulator must always begin at zero for each MAC computation.

2. **Block Canonicalization**

Message parsing into 32-byte blocks must be unambiguous, with zero padding of the final block.

3. **Fixed Subkeys**

The subkeys H and F must remain constant throughout a single MAC operation.

4. **Non-Exposure of Internal State**

The accumulator and subkeys must not be exposed externally. Only the final masked value is returned.

5. **Identical Behavior Across Architectures**

The intended design is platform neutral. Scalar, SIMD, and specialized instruction paths must all compute mathematically identical results.

These invariants reflect the designer's expectations, not the implementation specifics. Later chapters evaluate whether the code maintains these invariants in practice.

Chapter 4: Code Architecture and Module Overview

4.1 Overview

The QMAC implementation resides within a small and clearly defined portion of the QSC library. Its structure reflects an intentional effort to keep the MAC construction self-contained, free of hidden global state, and isolated from unrelated components. This chapter describes how the code is organized, how its primary modules interact, and how the internal state is represented and manipulated throughout the lifecycle of a QMAC computation.

The goal here is not to analyze correctness or security, but simply to document the architecture that supports the later stages of the analysis. Understanding this structure provides necessary context for examining the details of initialization, message processing, field arithmetic, and finalization in later chapters.

4.2 Source File Organization

The implementation is divided between two files:

- **qmac.h** defines the public API, the QMAC state structure, and the parameter structure used to configure initialization.
- **qmac.c** contains all executable logic, including initialization, update, finalization, field arithmetic, and state disposal.

This separation mirrors conventional C cryptographic interfaces: the header file exposes a narrow API surface with well-defined structures, while the source file encapsulates the operational details. There is no inter-module dependency beyond interactions with Keccak, cSHAKE, and memory utilities, each of which is accessed through stable and predictable interfaces.

4.3 QMAC State Structure

The core data structure is `qsc_qmac_state`, defined in `qmac.h`. It contains:

- storage for the raw key supplied at initialization,
- storage for the derived subkeys H and F ,
- storage for the accumulator Y ,

- a structure describing initialization parameters (nonce, info, mode),
- and a flag indicating whether the context is ready for use.

All fields are fixed in size. No variable-length buffers, heap allocations, or dynamically sized components appear in the structure. This simplifies memory management and reduces the risk of side effects or uninitialized behavior.

The state is designed for single-use MAC computations. Once a tag is produced, the caller is expected to dispose of the state and reinitialize it before processing another message.

4.4 Functional Modules Within qmac.c

The source file organizes the implementation into distinct logical modules. They are not separated into physical files, but can be understood as conceptual components:

4.4.1 Initialization Module

The initialization logic builds a cSHAKE state, absorbs the keying parameters, derives the subkeys, clears temporary buffers, and prepares the QMAC context for use. It sets the accumulator to zero and marks the context initialized.

4.4.2 Update and Message Parsing Module

The update function parses input messages into canonical 32-byte blocks. Blocks are copied into a temporary buffer and padded with zeros if necessary. This module handles block alignment but does not modify the accumulator directly.

4.4.3 Polynomial Accumulation Module

The block-level update routine handles the core MAC logic. It XORs the temporary block with the accumulator and then multiplies the result by H in the field $GF(2^{256})$. The multiplication and reduction routines are located nearby and are invoked directly.

4.4.4 Finalization Module

The finalization step masks the accumulator with F and writes the result to the caller-supplied tag buffer. It performs no additional computation and does not alter the subkeys or internal parameters.

4.4.5 Disposal Module

Disposal clears the accumulator, subkeys, raw key material, and parameters from memory. It resets the initialization flag and leaves the state structure sanitized for reuse or deallocation.

4.5 Interaction with Supporting Modules

QMAC relies on external modules within the QSC library, but these interactions are tightly scoped and predictable:

- **Keccak / cSHAKE** are used only during initialization, and all sponge state is cleared immediately after subkeys are extracted.
- **Memory utilities** provide constant-time XOR, clearing, copying, and a unified interface for scalar and SIMD carryless multiplication.
- **No external randomness source** is used, consistent with QMAC's deterministic design.

Each dependency is invoked explicitly. No hidden or implicit coupling exists between QMAC and other cryptographic modules, and there is no cross-component state sharing.

4.6 Architectural Intent and Design Discipline

The code architecture reflects a disciplined approach: every stage of the MAC computation has a single, well-defined place in the source. Initialization does not overlap with message parsing, field arithmetic does not interact with the cSHAKE wrapper, and finalization does not modify internal fields. This separation of concerns makes the implementation easier to reason about and reduces the risk of unintended interactions.

The next chapter builds on this structure to examine subkey derivation in detail.

Chapter 5: Subkey Derivation via cSHAKE

5.1 Overview

The subkey derivation process is the foundation upon which the remainder of the QMAC computation depends. Every authentication operation begins by generating two internal 256-bit values, H and F, through a single invocation of cSHAKE. These values

influence the polynomial hashing layer and the final masking step, respectively. If subkey derivation is flawed, inconsistent, or misaligned with the intended construction, all downstream guarantees collapse.

This chapter examines the derivation of H and F from the standpoint of specification intent, extension of SP 800-185 customization rules, and exact implementation behavior as encoded in the updated `qmac.c`.

5.2 Role of cSHAKE in QMAC

cSHAKE serves as a customizable, domain-separated extendable-output function. Its inclusion in QMAC provides three essential properties:

- A unified mechanism to absorb secret key material together with nonce and info inputs.
- A deterministic but flexible method for deriving subkeys under distinct protocol contexts.
- A pseudorandom stream large enough to extract disjoint internal keys without additional state mixing.

The design intent is that cSHAKE's domain separation capability be used to bind QMAC to protocol-level semantics, preventing unintended key or tag collisions between logically distinct uses of the MAC.

5.3 Absorption of Keying Parameters

Initialization begins with the caller supplying three classes of parameters:

- a secret key that provides cryptographic strength,
- an optional nonce used for structural separation of independent contexts,
- and an optional info field that may encode metadata, roles, or protocol semantics.

The implementation passes these values directly into the cSHAKE absorption phase. No intermediate hashing or reprocessing occurs prior to absorption; they are concatenated as raw byte strings, preserving exact caller intent.

5.3.1 Customization Constraints

QMAC's customization values, as carried into cSHAKE, ensure that:

- tags produced under different modes (256 vs 512) cannot collide,
- nonce and info values become part of the internal domain,
- the user-provided key is required to influence every output bit of H and F.

This behavior is consistent with SP 800-185, which mandates that customization act as part of the domain separation layer.

5.4 Subkey Extraction Process

After absorption, the cSHAKE instance performs a single squeeze operation. The implementation allocates a fixed 64-byte buffer and extracts one full rate-sized block, regardless of the mode. This block is partitioned to obtain:

- the first 32 bytes, which become the hashing key H,
- the next 32 bytes, which become the finalization key F.

This mechanism ensures that H and F are independent slices of the same pseudorandom output. Their independence is guaranteed by the sponge's internal diffusion and the uniformity of its output distribution under keyed initialization.

5.4.1 Rationale for Using a Single Squeeze

The design chooses a single squeeze for three reasons:

1. It minimizes the complexity of the derivation process.
2. It guarantees that both subkeys are bound to exactly the same domain-separation inputs.
3. It avoids redundant permutations or potential timing variance between separate absorb-squeeze cycles.

This decision closely aligns with the principle of minimal surface area in cryptographic engineering.

5.5 Clearing of Intermediate Material

Upon completion of subkey extraction, the implementation clears all temporary state used during derivation:

- the temporary cSHAKE output buffer,
- the internal Keccak state,
- and any residual absorption structures.

This behavior limits exposure of keying material and prevents inadvertent reuse of ephemeral intermediate values. It also ensures that no sensitive state persists beyond the initialization phase, which is consistent with good cryptographic hygiene.

5.6 Alignment with Design Intent

The implemented subkey derivation process conforms to the original design intent in all relevant respects:

- It uses cSHAKE's customization features correctly.
- It cleanly separates functional roles for H and F.
- It avoids hidden dependencies or implicit state sharing.
- It maintains determinism and reproducibility for testing.
- It provides a direct, transparent mapping from keying parameters to internal working keys.

This derivation model preserves the mathematical structure required for polynomial MAC security and provides a stable foundation for the remaining operational phases of QMAC.

Chapter 6: Message Processing Flow

6.1 Overview

The message processing phase defines how QMAC transforms an input message into a sequence of structured field elements and applies the hashing recurrence that ultimately determines the authentication tag. Every property of the MAC that relates to determinism, universality, and unforgeability emerges from this phase. For this reason, message processing must reflect the specification exactly and must be implemented in a way that is free from ambiguity, hidden assumptions, or architecture-dependent behavior.

This chapter describes how messages are parsed, how blocks are formed, how the accumulator evolves, and how the implementation ensures stable and repeatable behavior across platforms.

6.2 Block Structure and Parsing

QMAC processes messages in fixed 32-byte units, each of which is interpreted as an element of the field $\text{GF}(2^{256})$. The choice of 32 bytes is deliberate: it matches the width of the cSHAKE-derived subkeys and the polynomial basis of the field. This uniform block width simplifies both the accumulator recurrence and the field arithmetic.

Messages that are not multiples of 32 bytes are handled through a straightforward padding rule. The final, possibly partial block is copied into a temporary buffer and padded with zeros to fill the remaining bytes. This method avoids the need for a separate length field or sentinel value and guarantees that every concrete message maps deterministically to a well-defined sequence of field elements.

The implementation does not compress, re-encode, or otherwise transform message bytes. The mapping from bytes to field elements is direct and injective, a critical property for ensuring that two distinct messages do not share a polynomial representation.

6.3 Accumulator Initialization

Before processing begins, the accumulator Y is set to the zero element of the field. This is consistent with the mathematical definition of polynomial evaluation, in which the initial value plays no independent role and simply prepares the state for successive updates. The implementation initializes Y explicitly; it does not rely on implicit structure or uninitialized memory.

A fresh accumulator is required for every QMAC invocation. Reusing the accumulator across multiple messages would create cross-message interference and invalidate the security model, but the implementation avoids this by linking accumulator lifetime to the context lifecycle rather than to message segments.

6.4 Per-Block Update Function

The core of the message processing phase is the accumulator recurrence

$$Y \leftarrow H \cdot (Y \oplus X),$$

which is applied once per block. The update function in the implementation reflects this directly. It begins by XORing the block X with the current accumulator value Y . The result is treated as an element of $GF(2^{256})$. The function then invokes the internal multiplication routine with this value and H as inputs, reducing the product modulo the irreducible polynomial that defines the field.

This sequence mirrors the specification precisely. There are no side conditions, no data-dependent branches, and no deviations from the formal recurrence. Each block contributes linearly to the final accumulator, and the linear map determined by H remains constant throughout the computation.

6.4.1 XOR Operation

The XOR operation is implemented through a fixed-size memory utility that operates on 32-byte buffers. Its behavior is predictable and constant time, providing a stable mixing of block data with the accumulator.

6.4.2 Field Multiplication

Field multiplication relies on the internal polynomial multiplication routine described in the next chapter. It is critical here only to note that the multiplication always produces a valid field element and that the accumulator remains within $GF(2^{256})$ after each update.

6.5 Deterministic Message Interpretation

QMAC is deterministic by design, and the message processing logic reflects this property. Every message is interpreted in exactly the same way across runs and across architectures. No architectural feature, such as pointer alignment or endianness, affects the encoding of blocks. Byte ordering is preserved throughout and remains the same on all supported systems.

This deterministic behavior ensures that QMAC can be validated easily and that test vectors remain stable across different platforms, compiler settings, and optimization levels.

6.6 Consistency with Specification Intent

The implementation satisfies all major requirements associated with the intended behavior of the message processing phase:

- Each message is mapped uniquely and injectively to a sequence of 256-bit field elements.
- The accumulator recurrence strictly follows the polynomial evaluation model.
- Block boundaries are treated consistently, including padding of the final block.
- No block is skipped, reordered, merged, or combined implicitly.
- No internal state escapes or leaks through intermediate outputs.

The processing flow therefore aligns with the specification both in structure and detail.

6.7 Summary

Message processing in QMAC is straightforward but foundational. The implementation adheres closely to the specification and maintains the invariants upon which polynomial MAC security depends. The clear separation between parsing, accumulation, and field arithmetic contributes to a transparent and auditable codebase that behaves consistently across environments.

The next chapter examines the field arithmetic that powers the accumulation process.

Chapter 7: Field Arithmetic and Polynomial Basis

7.1 Overview

The security and correctness of QMAC depend on the behavior of its polynomial hashing layer, which in turn requires a well-defined binary field of size 2^{256} . Every block update involves a multiplication of two field elements followed by modular reduction. If this arithmetic is inconsistent, incorrect, or based on a reducible modulus, the entire MAC construction is weakened or invalidated. For this reason, the internal field representation is one of the most critical components of the implementation.

This chapter describes the polynomial basis selected for QMAC, the irreducible polynomial that defines the field, and the mechanics of multiplication and reduction as implemented in the updated version of `qmac.c`.

7.2 Representation of Field Elements

Each field element is represented as 256 bits arranged in little-endian byte order. This is consistent with the wider design of QSC, which treats byte strings as little-endian when converted to polynomial form. The representation is transparent and does not involve any hidden state transformations or encoding adjustments.

A 32-byte message block maps directly to a field element without preprocessing. This ensures that the mapping from input message bytes to field elements is one-to-one and architecture independent. Endianness is handled explicitly and consistently in all arithmetic operations.

7.3 Choice of Irreducible Polynomial

The implementation uses the irreducible pentanomial:

$$m(x) = x^{256} + x^{10} + x^5 + x^2 + 1.$$

This polynomial satisfies the necessary algebraic criteria:

- it is irreducible over $\mathbb{F}_2$,
- it ensures the existence of multiplicative inverses for all non-zero field elements,
- and it prevents the appearance of zero divisors, which would undermine the universality of the polynomial hash.

The selection of this polynomial restores the theoretical alignment between the implementation and the underlying MAC model.

7.4 Polynomial Multiplication

Multiplication in $\text{GF}(2^{256})$ is performed as polynomial multiplication over $\mathbb{F}_2$, yielding a 512-bit intermediate product. This computation is implemented through one of two paths:

- a carryless multiplication routine that uses SIMD intrinsics where available, or
- a scalar fallback routine that produces the same result without relying on hardware acceleration.

Both paths implement polynomial multiplication, not integer multiplication, and both produce identical outputs. The product is stored in an 8-word (64-bit per word) buffer representing degrees 0 through 511.

7.4.1 Platform Independence

Because the multiplication step is implemented using deterministic bitwise operations, differences in platform, compiler, or vector width do not influence correctness. Regardless of architecture, the product is always a strictly polynomial result suitable for modular reduction.

7.5 Reduction Modulo the Irreducible Polynomial

After multiplication, the 512-bit product is reduced modulo $m(x)$. The reduction process eliminates all terms of degree 256 or higher by applying the defining relation:

$$x^{256} = x^{10} + x^5 + x^2 + 1.$$

For each bit of the product above degree 255, the implementation:

1. identifies the bit,
2. clears it,
3. determines its relative offset from degree 256,
4. and XORs contributions into positions shifted by 0, 2, 5, and 10.

This operation is repeated for each high-order bit from degree 511 down to 256, ensuring that the final result lies entirely within the field.

7.5.1 Rationale for the Bit-Level Reduction Chain

Although not the fastest possible reduction method, the bit-level elimination process has two important virtues:

- it is transparent and auditable, allowing verification without complex vector logic,
- and it produces exactly the field element defined by the pentanomial, with no risk of architectural divergence.

This approach was deliberately chosen in the updated implementation to guarantee correctness.

7.6 Canonical Form and Field Correctness

At the end of the reduction process, the lower 256 bits of the intermediate buffer represent the final field element. This value becomes the new accumulator state in the polynomial hash. The implementation ensures that:

- no unreduced high-order terms remain,
- the accumulator always resides within $\text{GF}(2^{256})$,
- and identical inputs yield identical field elements across platforms.

These invariants are essential for maintaining the universality and collision-resistance properties expected of polynomial hashes in large fields.

7.7 Summary

QMAC's field arithmetic is consistent with a correct and secure polynomial MAC model.

The multiplication and reduction routines are deterministic, architecture-independent, and auditable. Their behavior aligns fully with the mathematical expectations established in earlier chapters and provides the necessary foundation for the correctness of the entire MAC construction.

The next chapter extends this analysis to the behavioral structure of the internal functions that depend on these arithmetic operations.

Chapter 8: Internal Function Behavior and Data Movement

8.1 Overview

QMAC's behavior emerges from a small set of internal routines that coordinate message parsing, accumulator updates, field arithmetic, and finalization. Each function plays a specific role in enforcing the structural integrity of the MAC, and each contributes to the disciplined separation between phases that we observed in earlier chapters. This chapter examines the responsibilities of these internal functions and describes how they manipulate state within the QMAC context.

The intention here is not to analyze correctness from a cryptanalytic perspective—that task is reserved for later chapters—but to ensure that each function's behavior is well understood and consistent with the structural specification described in the earlier sections of this document.

8.2 qmac_block_update

The `qmac_block_update` routine orchestrates the core accumulator update. It accepts a 32-byte block that has been prepared by the message parsing logic and performs two steps:

1. It XORs the block with the current accumulator Y .
2. It multiplies the result by the subkey H within the field.

These two operations correspond exactly to the recurrence defined in the QMAC construction. There is no deviation, hidden transformation, or dependence on message length or position within the message. Every block, including the final padded one, undergoes the same update.

8.2.1 Data Movement

The function does not allocate memory or create additional copies of the block. All operations work directly on fixed-size arrays within the context or on a small stack-allocated buffer passed to the routine. This approach reduces the surface area for unintended state leaks and maintains a predictable and auditable data flow.

8.3 qmac_gfmul256_poly

The multiplication routine is responsible for computing $H \cdot Z$ where $Z = Y \oplus X$. As described in Chapter 7, the function first computes the polynomial product and then reduces it modulo the irreducible polynomial. From the standpoint of function behavior:

- The function always returns a well-formed element of the field.
- It performs no validation of the inputs, operating under the assumption that the caller provides two 256-bit values.
- It produces identical results on scalar and SIMD implementations.

This routine is the most computationally expensive part of QMAC but also one of the simplest to verify, as the arithmetic is explicit and does not contain data-dependent branching.

8.3.1 Control Flow Discipline

The function always reduces all degrees above 255. Even if the high 256 bits of the product are all zero; a rare but possible event, the reduction loop still executes. This ensures consistent run time and avoids conditional behavior based on secret values.

8.4 qmac_compute_final

Once all blocks have been processed, the finalization function computes the authentication tag by XORing the accumulator Y with the subkey F. The routine then copies the resulting 32-byte value into the caller-provided output buffer.

There are no additional dependencies or side effects. The accumulator is not modified after finalization, but it becomes irrelevant once the tag has been emitted, as the caller is expected to dispose of the context immediately afterward. The function does not clear internal state; that responsibility lies with the disposal routine.

8.4.1 Tag Generation Consistency

Because the tag is a pure XOR of two internal values, and because both values are 256 bits, the tag is always exactly 32 bytes. No variable length modes or truncated tags exist within this implementation.

8.5 Memory Utility Interactions

QMAC uses a small set of memory utilities for fundamental operations such as copying, XORing, clearing, and performing carryless multiplication. These utilities guarantee: fixed-size, constant-time behavior, predictable data movement, and architectural uniformity.

The internal functions rely heavily on these utilities instead of open-coded logic. This contributes to a consistent implementation style and reduces the risk of subtle bugs that might arise from manually managing low-level byte operations.

8.6 Life Cycle and State Boundaries

Internal functions operate strictly within the boundaries of the QMAC state. None of them access external hidden state, global buffers, or static memory. The initialization function sets up the state, the update function modifies it, the multiplication routine transforms intermediate values, and the finalization function extracts the tag.

Once the tag is extracted, the disposal function clears all sensitive fields; including the raw key, subkeys, accumulator, and parameter structures, and resets the initialization flag. This ensures that no state persists into subsequent computations.

8.7 Summary

The internal functions of QMAC demonstrate a disciplined design. Each routine is narrow in purpose, avoids hidden behavior, and manipulates data only within carefully controlled boundaries. The block update routine implements the accumulator recurrence accurately, the multiplication routine provides correct finite-field arithmetic, the finalization routine applies the required masking step, and the disposal routine ensures that no sensitive information remains after use.

This modularity and clarity greatly simplify verification and contribute to a robust and auditable MAC implementation. The next chapter examines how these functions and the implementation as a whole, align with the mathematical model and security properties expected of the construction.

Chapter 9: Alignment With the Mathematical Model

9.1 Overview

The purpose of this chapter is to verify that the QMAC implementation behaves in a manner consistent with the mathematical construction presented in earlier chapters. While the previous sections examined specific routines in isolation, this chapter considers the system holistically. The central question is whether the implementation's subkey derivation, message processing rules, field operations, and finalization step collectively realize the intended polynomial MAC.

This analysis addresses alignment at the conceptual level. Security guarantees such as universality, resistance to collision-based attacks, and the masking of internal state depend fundamentally on the faithful implementation of the mathematical model.

9.2 Subkey Derivation Consistency

The mathematical model for QMAC requires two independent 256-bit subkeys, H and F , derived from a single pseudo-random generator seeded with a secret key and contextual inputs. The implementation achieves this by invoking cSHAKE once,

absorbing the key, nonce, and info fields, and extracting 64 contiguous bytes of output. The first half becomes H; the second becomes F.

This approach satisfies the requirements of the model for three reasons. First, cSHAKE acts as a pseudo-random function under keyed initialization, ensuring that H and F are unpredictable to an adversary. Second, deriving both subkeys from a single-block guarantees that they share the same domain-separation context without requiring additional permutations. Third, the extraction is deterministic, facilitating reproducibility and simplifying analysis without reducing security.

No additional transformations are applied to the subkeys. The implementation therefore aligns precisely with the expectation that H and F be independently distributed 256-bit values produced by a uniform and domain-separated pseudorandom source.

9.3 Message Encoding and Canonicalization

The mathematical description of QMAC interprets a message as a sequence of elements in $\text{GF}(2^{256})$, obtained by partitioning the byte string into 32-byte blocks and padding the final block with zeros if necessary. The implementation adheres strictly to this encoding.

The absence of length fields, delimiters, or variable-size parsing ensures that the byte sequence maps deterministically to field elements. This injectivity is required for the proof that collisions in the polynomial hash occur only with negligible probability under randomly chosen H. Any deviation from this encoding model would introduce ambiguity; the implementation contains none.

Furthermore, because the encoding does not depend on platform endianness or alignment, the mathematical requirement of platform uniformity is preserved.

9.4 Accumulator Recurrence and Polynomial Evaluation

The core recurrence in QMAC computes:

$$Y_{i+1} = H \cdot (Y_i \oplus X_i),$$

where each X_i is a field element derived from a message block. This recurrence is the defining structure of the polynomial MAC. The implementation matches it exactly.

The XOR operation uses byte-wise exclusive OR, which corresponds to addition in the binary field. The multiplication is polynomial multiplication over $\mathbb{F}_2$ followed by modular

reduction, ensuring that each update maps the accumulator to another valid field element in $\text{GF}(2^{256})$. No additional operations are introduced, and none are omitted.

Because the accumulator starts at zero and the recurrence is applied exactly once per block, the implementation produces the same final accumulator that the mathematical model predicts.

9.5 Finite-Field Arithmetic Correctness

A key requirement for alignment is the correctness of finite-field multiplication. The mathematical model assumes arithmetic in the field defined by an irreducible modulus. The implementation uses:

$$m(x) = x^{256} + x^{10} + x^5 + x^2 + 1,$$

which is irreducible over $\mathbb{F}_2$. By reducing the 512-bit product using this polynomial, the code ensures that multiplication is closed and that every nonzero element has an inverse.

This satisfies the algebraic invariants needed for polynomial MAC security. The reduction process is explicit, exhaustive, and independent of message content, guaranteeing consistency with the intended mathematical behavior across all architectures.

9.6 Final Masking and Exposure Control

The mathematical definition of QMAC dictates that the accumulator not be exposed directly. Instead, it is masked with a second subkey F to produce the final tag:

$$T = Y \oplus F.$$

The implementation performs this operation exactly. No intermediate or partial accumulator states are leaked, and no additional transformations are applied to either operand. This alignment is essential to prevent algebraic attacks that could exploit the raw accumulator.

Because both Y and F are 256-bit values, the tag is always a full 256-bit quantity. The implementation's fixed-length output corresponds directly to the constant-size tag defined by the model.

9.7 Architectural and Platform Alignment

The mathematical model requires that QMAC behave identically on all supported platforms. The implementation satisfies this requirement:

- Field multiplication is deterministic regardless of whether SIMD or scalar routines are used.
- Block parsing is invariant under endianness.
- cSHAKE initialization uses explicit byte ordering.
- The accumulator and subkeys are stored in architecture-neutral layouts.

The resulting behavior is consistent across differing compilers, processor architectures, and optimization levels.

9.8 Summary

Across its entire lifecycle; from subkey derivation to message encoding, polynomial evaluation, and final masking, the QMAC implementation reflects the mathematical model faithfully and without deviation. The alignment is exact, and the updated irreducible polynomial restores full consistency between code and theory.

The next chapter examines how these aligned behaviors relate to security-relevant implementation properties, including timing uniformity, memory handling, and exposure resistance.

Chapter 10: Security-Relevant Implementation Properties

10.1 Overview

Security-relevant behavior in a message authentication code arises not only from the algorithm's mathematical properties but from the way the implementation handles data, processes instructions, and exposes or conceals internal state. QMAC's correctness as a polynomial MAC depends on the consistency of its finite-field arithmetic, the predictability of its control flow, and the disciplined handling of sensitive material throughout initialization, message processing, and disposal. This chapter evaluates the implementation from this perspective, identifying the operational properties that influence security and confirming their alignment with the intent of the design.

10.2 Timing Behavior and Control-Flow Predictability

A key property of the QMAC implementation is the absence of secret-dependent control flow. No routine makes branching decisions based on the values of the key, the subkeys, or the accumulator. In particular:

- The polynomial reduction loop in the multiplication routine executes a fixed sequence of iterations regardless of operand values.
- The XOR operations behave uniformly across all 32 bytes of the input.
- Subkey extraction uses a fixed amount of cSHAKE output, independent of message content.

Although the reduction loop is long, its length is constant and does not vary with secret material. Therefore, timing differences reflect only message length and platform performance characteristics. This predictability is essential for resisting timing-based attacks, and the implementation meets this requirement.

10.3 Memory Safety and Clearing of Sensitive Data

The QMAC implementation relies exclusively on fixed-size arrays; it performs no heap allocation and does not manipulate pointers through arithmetic or dynamic structure. This approach eliminates classes of memory safety vulnerabilities that arise from unmanaged buffers or variable-length structures.

Sensitive data, including the raw key, subkeys, accumulator, and temporary cSHAKE output, is explicitly cleared as soon as it is no longer required. The clearing routines operate on known buffer sizes and do not rely on compiler optimizations or inline expansion to maintain correctness. This consistent, explicit clearing ensures that residual sensitive material does not remain in memory after use.

10.4 Deterministic Operation and Equality Leakage

QMAC is intentionally deterministic: the same keying parameters and message always produce the same tag. The implementation faithfully preserves this property. While determinism is appropriate for a polynomial MAC, it also introduces a limited form of equality leakage. An observer who sees multiple tags under the same key and domain parameters can infer whether two messages are identical. The implementation neither mitigates nor amplifies this property; it is inherent in the design. Protocols requiring resistance to equality leakage must incorporate additional randomness at higher layers.

10.5 Subkey Integrity and Independence

The integrity of H and F is central to QMAC's security. Both subkeys must remain internal, unpredictable, and independent:

- The implementation derives both subkeys from the same cSHAKE output block, ensuring they share the same domain-separation context.
- They occupy disjoint regions of that output, ensuring independence through the properties of the sponge construction.
- They are never exposed externally, logged, or returned to the caller.

Throughout message processing and finalization, the implementation uses H and F in their intended roles and avoids unnecessary manipulation or duplication of subkey material.

10.6 Exposure Surface and Internal-State Containment

The accumulator Y is updated after each block but is never exposed through the API. Only the final masked value, obtained by XORing Y with F, is returned. The masking step ensures that internal accumulator values cannot be reconstructed directly from tags, even across multiple messages.

No intermediate field products or unreduced values leak outside the multiplication function. Similarly, the cSHAKE absorption and squeezing process does not leave residual unmasked output in the context. The exposure surface is therefore limited to:

- the final 32-byte tag,
- the explicit parameters supplied by the caller,
- and the resolution of success or failure flags.

This controlled exposure corresponds precisely to the intended design.

10.7 Architectural Portability and Behavioral Invariance

The implementation is designed to behave identically across scalar and SIMD architectures:

- SIMD acceleration, when present, affects only the speed of the carryless multiplication routine, not the mathematical result.

- The reduction process is fully architecture-independent.
- Byte ordering is controlled explicitly through fixed-size operations.

These choices ensure reproducibility and cross-platform consistency, both essential properties for a cryptographic primitive.

10.8 Alignment with Security Expectations

Across all stages; initialization, accumulation, multiplication, and finalization, the implementation adheres to the design's security expectations:

- No internal state leaks.
- No control-flow deviations arise from secret data.
- No dynamic memory hazards threaten stability.
- Deterministic behavior is preserved intentionally.
- The accumulator and subkeys are bounded within the QMAC state and cleared on disposal.

These properties collectively support the construction's integrity and provide strong confidence in the security posture of the implementation.

10.9 Summary

The security-relevant behavior of the QMAC implementation aligns closely with the mathematical model and the design intentions described earlier in this document. Its timing profile, memory safety, subkey handling, and exposure controls all reflect disciplined cryptographic engineering. The implementation introduces no observable deviations that would undermine correctness or security. These findings form the basis for the broader security assessment developed in later chapters.

Chapter 12: Correctness Verification and Reproducibility

12.1 Overview

Reproducibility is essential for any cryptographic primitive that aims to be verifiable, testable, and portable across platforms. QMAC's deterministic nature lends itself to consistent behavioral testing, but correctness also depends on ensuring that every stage

of the computation, subkey derivation, block processing, field arithmetic, and finalization produces results identical to those expected from the mathematical model. This chapter examines the degree to which the implementation supports reproducibility and outlines the procedures available for independent verification of correctness.

12.2 Deterministic Output and Behavioral Stability

QMAC produces deterministic outputs for a given set of keying parameters and a message. The implementation does not incorporate randomness at any point in the computational flow. This determinism is a deliberate design choice, making it straightforward to validate: correctness of the field arithmetic, consistency of block processing, and stability of cSHAKE-derived subkeys.

Because the construction does not rely on per-message or per-block randomness, the same input always yields the same tag across architectures, compiler settings, and optimization levels. This stability is a cornerstone of reproducibility.

12.3 Test Vector Generation

Test vectors play an important role in verifying correctness. They confirm that two independent implementations derive identical keys, interpret messages consistently, and perform field arithmetic identically. Creating such vectors with QMAC is straightforward:

1. Assemble a known key, nonce, and info set.
2. Process a fixed test message.
3. Output the resulting accumulator mask (the final tag).
4. Repeat the process under scalar and SIMD builds to confirm architectural independence.

The implementation's deterministic block processing ensures that any deviation between builds or environments is easy to detect.

12.4 Independent Verification of Field Arithmetic

The correctness of the polynomial multiplication routine and reduction logic can be verified independently of the rest of the QMAC implementation. A small, language-agnostic reference implementation of multiplication modulo

$$x^{256} + x^{10} + x^5 + x^2 + 1$$

can be used to test the field arithmetic. The verifier can: generate random 256-bit values for H and Z, compute the product and reduction using the reference implementation, compare the result with the output produced by `qmac_gfmul256_poly`.

Because the reduction logic in the implementation follows a transparent, exhaustive bit-level elimination procedure, verification is simple and does not depend on architectural features.

12.5 Subkey Derivation and cSHAKE Reproducibility

Verification of the subkey derivation procedure requires confirming that H and F are extracted correctly and that cSHAKE is invoked in accordance with SP 800-185. Given a known key, nonce, and info field, an independent cSHAKE implementation must produce: a rate-sized output block identical to the one used by QMAC, and subkey bytes that match the implementation's extraction of H and F.

The implementation clears the cSHAKE state after derivation but does not alter the extracted subkeys. This makes cross-checking trivial.

12.6 Cross-Architecture Consistency

One of QMAC's strengths is its design for identical behavior across architectures. Architectural independence derives from several factors: The polynomial multiplication is defined in pure binary algebra.

- The reduction routine executes identically regardless of the availability of SIMD instructions.
- Block parsing and byte interpretation are platform-agnostic.
- The output of cSHAKE remains consistent across implementations as long as SP 800-185 is followed.

Testing under diverse compilers and platforms should reveal no functional differences beyond performance variation.

12.6 Reference Implementation Equivalence

Because of QMAC's simple structure, full equivalence testing can be performed using a reference implementation in a high-level language such as Python or Rust. This involves: replicating the cSHAKE absorption process, implementing the finite-field multiplication

and reduction, computing the same polynomial recurrence, and verifying that the final tags match the ones emitted by the C implementation.

A reference implementation enables continuous regression testing and helps ensure that future optimizations or refactorings do not drift from the mathematical model.

12.7 Summary

QMAC exhibits a strong alignment between its mathematical model and implementation, enabling comprehensive reproducibility and correctness verification. Its deterministic nature, fixed-size operations, and transparent field arithmetic facilitate: direct test vector generation, independent verification across platforms, and architectural consistency of results.

These properties contribute to the correctness and trustworthiness of the implementation and support long-term maintainability and validation.

Chapter 13: Limitations and Engineering Risks

13.1 Overview

Although the QMAC implementation is structurally sound and aligned with its mathematical model, every cryptographic primitive carries inherent limitations shaped by design choices, performance constraints, and expected deployment environments. This chapter evaluates those limitations from an engineering perspective and identifies areas where careful protocol-level handling or future refinement may be necessary.

13.2 Deterministic Operation and Protocol-Level Risks

QMAC is intrinsically deterministic. For a fixed key, nonce, and info field, identical messages always produce identical tags. This behavior is appropriate for a polynomial MAC, but it introduces privacy considerations in certain environments. An adversary capable of observing multiple tags under the same key can trivially test for message equality.

The implementation does not soften this determinism with randomization or message blinding. As a result, protocols that require unlinkability or resistance to equality leakage must add randomness or domain separation at higher layers. This limitation does not

arise from a flaw in the implementation but from the fundamental structure of deterministic MACs.

13.3 Dependence on Subkey Entropy and Parameter Integrity

The security of QMAC depends heavily on the entropy of the secret key and the correct usage of nonce and info parameters. The implementation does not validate the quality of these inputs. It accepts any caller-supplied bytes and treats them as structurally meaningful, trusting the protocol to provide sufficient entropy and appropriate domain separation.

Incorrect governance; for example, reusing the same key and identical customization parameters across unrelated protocols, can compromise security. This dependency on caller discipline is inherent to cSHAKE-based customization but must be acknowledged as a systemic limitation.

13.4 Scalar Performance Constraints

While the field arithmetic is correct and platform-neutral, the bit-level reduction loop introduces a visible performance cost on scalar architectures. Every multiplication processes all bits above degree 255, even when the high portion of the product is sparse. Although the loop is constant-time and easy to verify, it is not optimized for high throughput.

On systems without hardware carryless multiplication instructions, the performance gap may be significant. This limitation does not affect correctness but may restrict QMAC's suitability for high-throughput applications where many MAC computations must occur per unit time.

13.5 No Built-In Misuse Resistance

The implementation assumes correct usage by the caller. It allows multiple update calls after initialization, but does not enforce that each QMAC context is used exactly once. It does not forbid multiple finalization attempts. It does not track whether a context is being reused improperly.

Misuse resistance must therefore be provided by the calling protocol or framework. The design philosophy values simplicity and minimalism, but this lack of stateful enforcement introduces risks in environments where misuse is likely.

13.6 Limited Scope of Functionality

QMAC is intentionally narrow in function. It authenticates messages but does not provide: encryption, replay protection, ordered message sequencing, message bundling, or domain separation enforcement beyond cSHAKE absorption.

These features must be constructed outside the implementation. Systems that adopt QMAC must therefore rely on protocol design to avoid semantic gaps or misuse.

13.7 Architectural Portability and Optimization Tradeoffs

The implementation guarantees correctness across architectures but does not exploit platform-specific optimizations beyond a basic carryless multiplication interface. For example, the reduction algorithm could be improved on AVX2 or other SIMD-enabled systems through word-level folding or parallel reduction techniques. Such optimizations are absent in the current design.

This decision reflects a prioritization of transparency and simplicity, but it also limits peak performance in environments where optimized arithmetic is valuable. There is an opportunity for future refinement without altering core behavior.

13.7 Summary

QMAC's implementation is robust and correct, but it places certain responsibilities on the surrounding protocol and exhibits some natural limitations: determinism reveals message equality, key and parameter integrity rely on caller discipline, performance is constrained on scalar systems, and misuse resistance is not embedded.

These limitations are not design flaws but reflect deliberate tradeoffs in favor of clarity, consistency, and a minimal trusted computing base. Later chapters consider how these constraints influence long-term maintainability and suggest directions for future work.

Chapter 14: Conclusion

14.1 Overview

This chapter concludes the implementation analysis by synthesizing the findings of the preceding sections and evaluating the QMAC implementation as a whole. The analysis has reviewed the algorithm's intended construction, the code architecture, the behavior

of core routines, the correctness of field arithmetic, and the security-relevant properties that govern QMAC's operation. The resulting assessment reflects both the strengths of the implementation and the areas where careful governance or future refinement may be appropriate.

14.2 Summary of Findings

The QMAC implementation faithfully embodies the mathematical model of a deterministic polynomial MAC operating in $GF(2^{256})$. Subkey derivation is handled securely through a single cSHAKE invocation, message parsing is unambiguous and platform independent, and the accumulator recurrence aligns exactly with the theoretical construction.

The internal functions operate within disciplined boundaries, manipulating fixed-size state with predictable control flow. Sensitive material is cleared reliably, and no intermediate state escapes the confines of the QMAC context. The result is an implementation that is transparent, auditable, and consistent across architectures.

14.3 Security Posture

Security-relevant properties; including constant-time behavior, strict memory handling, deterministic block processing, and bounded exposure of internal state, align with expectations for a well-constructed MAC primitive. The implementation introduces no deviations that would compromise confidentiality or integrity under the assumed threat model.

The primary security considerations arise not from flaws in the implementation but from the inherent nature of deterministic MACs and the flexibility of cSHAKE customization. Protocols must govern keying material carefully, enforce appropriate domain separation, and avoid context reuse. When these responsibilities are met, the implementation's security posture is strong.

14.4 Engineering Considerations

The implementation is simple, maintainable, and platform neutral, but certain engineering tradeoffs remain:

- The bit-level reduction loop is correct and auditable but not optimized for high throughput on scalar systems.

- Deterministic operation produces equality leakage, requiring thoughtful integration into higher-level protocols.
- Misuse resistance is not built into the implementation and must be enforced externally.

These considerations do not diminish the quality or correctness of the implementation, but they guide future development priorities.

14.5 Long-Term Maintainability

Because QMAC relies on small, modular components with clearly defined roles, it is well positioned for long-term maintenance. The simplicity of the accumulator recurrence, the explicitness of the reduction algorithm, and the lack of dynamic memory allocation contribute to the clarity of the code and ease of auditing. Future improvements for example, optimized reduction routines or hardware-specific acceleration, can be added without altering the core structure of the implementation.

14.6 Final Assessment

The implementation of QMAC within the QSC library is correct, consistent with its specification, and engineered with clarity and discipline. It provides a reliable foundation for message authentication within systems that manage keying material and domain parameters appropriately. While certain performance and integration limitations are inherent to the design, the implementation achieves its intended purpose with a high degree of assurance.

Chapter 15: References

This section lists the authoritative standards, publications, and research papers referenced throughout this document. Each citation corresponds to a formally published or verifiable source that substantiates the mathematical constructions, cryptographic assumptions, and implementation methodologies used in the QMAC analysis.

National Institute of Standards and Technology *FIPS Publication 202: SHA-3 Standard – Permutation-Based Hash and Extendable-Output Functions*, August 2015.

URL: <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.202.pdf>

National Institute of Standards and Technology *NIST Special Publication 800-185: SHA-3 Derived Functions: cSHAKE, KMAC, TupleHash and ParallelHash*, December 2016.

URL: <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-185.pdf>

National Institute of Standards and Technology *NIST Special Publication 800-38D: Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*, November 2007.

URL: <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-38d.pdf>

McGrew, D. and Viega, J. *The Galois/Counter Mode of Operation (GCM)*, NIST Submission, 2004.

URL: <https://csrc.nist.gov/csrc/media/projects/block-cipher-techniques/documents/bcm/modes/proposed-modes/gcm/gcm-revised-spec.pdf>

Daemen, J. and Bertoni, G. *The Keccak Reference*, Submission to NIST SHA-3 Competition, 2011.

URL: https://keccak.team/keccak_specs_summary.html

Lidl, R. and Niederreiter, H. *Finite Fields*, Cambridge University Press, 1997. ISBN 978-0-521-45411-2.

Grover, L. K. *Quantum Mechanics Helps in Searching for a Needle in a Haystack*, *Physical Review Letters*, Vol. 79, No. 2, pp. 325–328, 1997.

Kocher, P., Jaffe, J., and Jun, B. *Differential Power Analysis*, *Advances in Cryptology – CRYPTO 1999*, Springer, 1999.

Bertoni, G., Daemen, J., Peeters, M., and Van Assche, G. *Keccak Implementation Overview*, Keccak Team Note, 2012.

URL: https://keccak.team/keccak_implementation.html

Ferguson, N., Kelsey, J., Lucks, S., Schneier, B., Whiting, D., Bellare, M., Kohno, T., Callas, J., and Walker, J. *The Skein Hash Function Family*, NIST SHA-3 Competition Submission, 2010.

Boneh, D. and Shoup, V., A. *Graduate Course in Applied Cryptography*, Version 0.6, Stanford University, 2020.

Bernstein, D. J. *Multidigit Multiplication for Mathematicians*, University of Illinois at Chicago, 2001.

Bernstein, D. J. and Lange, T. *Post-Quantum Cryptography*, *Nature*, Vol. 549, 2017, pp. 188–194.

Dworkin, M. *Recommendation for Block Cipher Modes of Operation: Methods and Techniques*, NIST Special Publication 800-38A, 2001.

Lenstra, A. K. and Verheul, E. R. *Selecting Cryptographic Key Sizes*, *Journal of Cryptology*, Vol. 14, 2001, pp. 255–293.

Bellare, M. and Rogaway, P. *Introduction to Modern Cryptography*, University of California, Davis, 2005.

Alagic, G., et al. *Status Report on the Second Round of the NIST Post-Quantum Cryptography Standardization Process*, NIST Internal Report 8309, 2020.

Matsui, M. *Linear Cryptanalysis Method for DES Cipher*, *Advances in Cryptology – EUROCRYPT '93*, Springer, 1994.

Biryukov, A., Dunkelman, O., Keller, N., Khovratovich, D., and Shamir, A. *Key Recovery Attacks of Practical Complexity on AES Variants with Up To 10 Rounds*, *ASIACRYPT 2010*.

National Institute of Standards and Technology *Recommendation for Key Management: Part 1 – General*, NIST Special Publication 800-57, Revision 5, May 2020.

URL: <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-57pt1r5.pdf>

ISO/IEC 19790:2012 *Information Technology — Security Techniques — Security Requirements for Cryptographic Modules*, International Organization for Standardization.

ISO/IEC 29100:2011 *Information Technology — Security Techniques — Privacy Framework*, International Organization for Standardization.

Killmann, W. and Schindler, W. *AIS 31: Functionality Classes and Evaluation Methodology for True Random Number Generators*, Federal Office for Information Security (BSI), Bonn, 2011.

Rogaway, P., Bellare, M., Black, J., and Krovetz, T. *OCB: A Block-Cipher Mode of Operation for Efficient Authenticated Encryption*, *ACM CCS*, 2001.

QRCS Corporation. *QSC Cryptographic Library - QMAC Implementation Files*.
GitHub Repository: <https://github.com/QRCS-CORP/QSC>